

MemCache-Ejercicio-4.pdf



Pausevi



Estructura de Computadores



2º Grado en Ingeniería Informática



**Facultad de Informática
Universidad Complutense de Madrid**

Ejercicio 4. Considere un computador con direcciones de 32 bits, al que se dota de una memoria cache de 128 bytes, con bloques de 16 bytes y emplazamiento directo, sin asignación en escritura (No write-allocate). Suponga que la variable a está en el banco de registros y no en cache.

Se quiere ejecutar el siguiente código:

```
int nota[128]; // nota[0] está almacenado en la dirección 0x00000000
int media[128]; // media[0] está almacenado a continuación de nota[127]
```

```
for (i=0;i<128;i++) {
    if (i>7 && i<64) {
        nota[i] = media[i]/2;
    }
    else {
        a = nota[i]*media[i]
    }
}
```

- a) Indique cuántos fallos de cache se producen.
- b) Proponga dos optimizaciones de código (independientes) que mejoren el resultado, indicando cuántos fallos se producen en esos casos.
- c) Si el tiempo de acceso a una palabra en cache es 1 ns, el tiempo de lectura o escritura de una palabra en memoria principal es 50 ns, el tiempo de transferencia de un bloque de MP a cache es 200 ns, ¿cuál es el tiempo necesario para acceder a todas las referencias del programa?
- d) Si se añade al computador una memoria cache de segundo nivel de 1MB y asociatividad 4, con bloques de 16 bytes, con asignación en escritura y actualización diferida y si el tiempo de transferencia de un bloque desde memoria cache de segundo nivel a la de primer nivel son 15ns, ¿cuál es el tiempo necesario para acceder a todas las referencias del programa?

1) Accesos a memoria

```
int nota[128]; // nota[0] is stored in address 0x00000000
int media[128]; // media[0] is stored after nota[127]
```

```
for (i=0;i<128;i++) {
    if (i>7 && i<64) {
        nota[i] = media[i]/2;
    }
    else {
        a = nota[i]*media[i]
    }
}
```

Desde i=8 hasta i=63 (56 iteraciones):

1 lectura en memoria (media) y
1 escritura en memoria (nota)

Las restantes 64+8 iteraciones:
2 lecturas en memoria (nota y media)

Los accesos a las instrucciones no se consideran en este ejercicio.

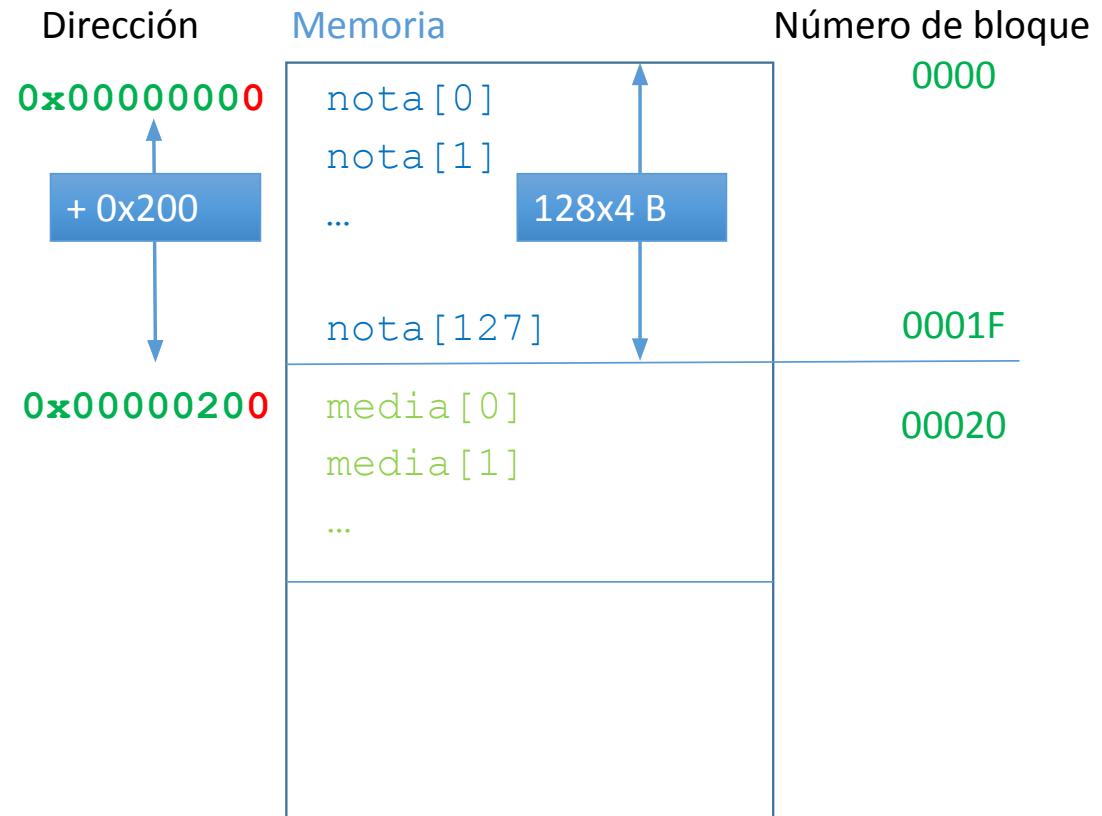
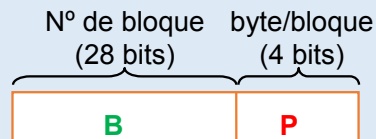
2) Mapa de bloques de memoria

Bloques de 16B

$$\frac{16 \text{ bytes/bloque}}{4 \text{ bytes/elemento}} = 4 \text{ elementos/bloque}$$

Cada vector necesita 32 bloques de memoria

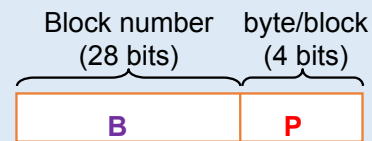
Campos de la dir. de memoria



3) Estructura de la cache

Cache de emplazamiento directo de **128B** con bloques de **16B**
Dónde se almacena en la cache media[0]?

Campos de la dir. de memoria



0x0000020 0

Etiqueta

 → Bloque de cache
0x000004 0b000

$$\frac{128 \text{ bytes}}{16 \text{ bytes/bloque}} = 8 \text{ bloques}$$

Dir. de memoria: 0x00000200

0x0000020

0000 0000 0000 0000 0000 0010 0 000

0x000004

Para el mismo valor de i nota y media compiten por el mismo bloque de cache.

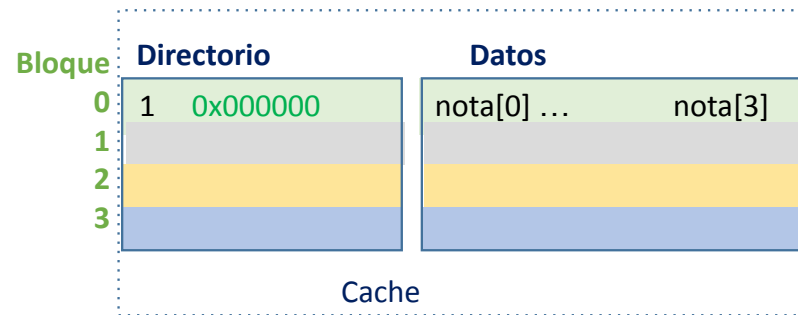
Accesos y fallos(1)

i=0

1. Lectura `nota[0]`

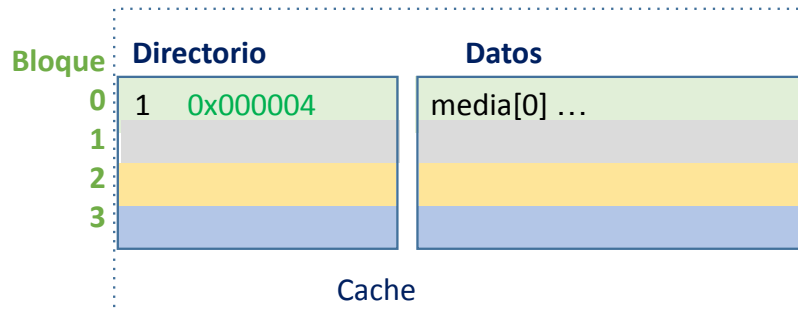
Fallo de cache

```
for i=0 to i=7 and for i=64 to 127  
  a = nota[i]*media[i]
```



2. Lectura `media[0]`

Fallo de cache



Iteraciones: $64 + 8 = 72$

Accesos: 2 por iteración
(lecturas)

Fallos: 2 por iteración
(fallos de lectura)

Conflicto:

2 bloques de memoria compiten por el mismo bloque de cache

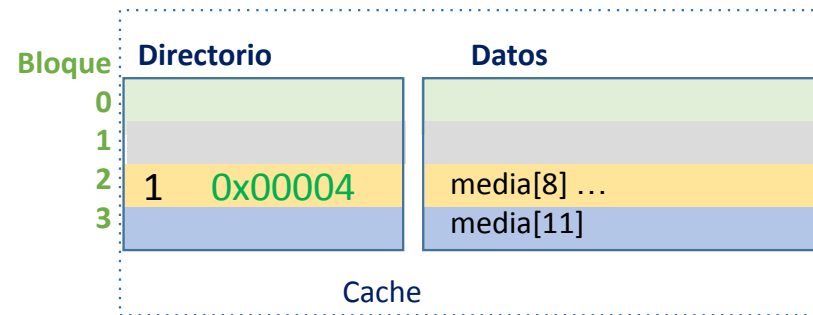
Accesos y fallos (2)

i=8

1. Lectura `media[8]`

Fallo de cache

```
for i=8 to 63
  nota[i] = 0,5*media[i]
```



2. Escritura `nota[8]`

Fallo de cache: no-write allocate

1 fallo de lectura por bloque.
Todas las escrituras son fallos (los elementos se escriben en memoria).

Iteraciones: $64 - 8 = 56$

Accesos: 1 lectura + 1
escritura por iteración

Fallos: 1 fallo de lectura
cada 4 iteraciones + 1 fallo
de escritura por iteración

a) Indique cuántos fallos de cache se producen

Iteraciones: $64 + 8 = 72$

Accesos: 2 per iteración

Fallos: 2 per iteración

Iteraciones: $64 - 8 = 56$

Accesos: 1 lectura + 1 escritura por iteración

Fallos: 1 fallo de lectura cada 4 iteraciones + 1 fallo de escritura por iteración

Acceses = $2 * 128 = 256$ (nº total de accesos)

Read_misses = $2 * 72 + 56/4 = 158$ (fallos de lectura)

Write_misses = $1 * 56$ (fallos de escritura)

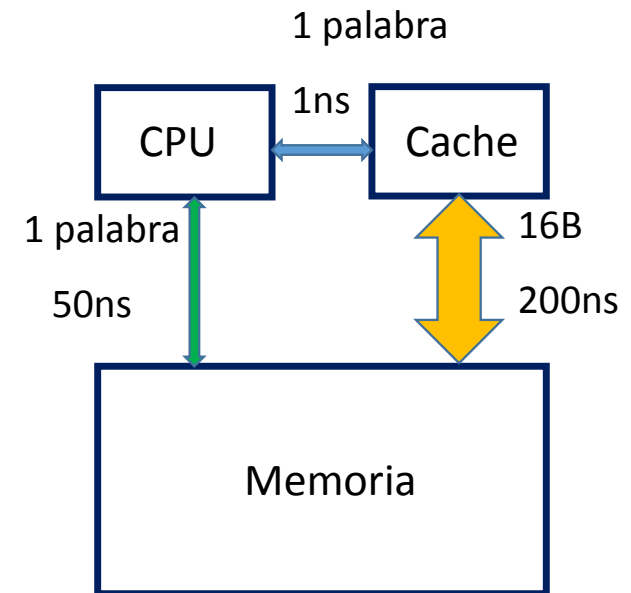
c) Si el tiempo de acceso a una palabra en cache es 1 ns, el tiempo de lectura o escritura de una palabra en memoria principal es 50 ns, el tiempo de transferencia de un bloque de MP a cache es 200 ns, ¿cuál es el tiempo necesario para acceder a todas las referencias del programa?

$$\text{Time_mem} = \# \text{Accesses} * T_{\text{hit}} + \# \text{Read_misses} * \text{Penalty_read} + \# \text{Write_misses} * \text{Penalty_write}$$

#Accesses = 256
#Read_misses = 158
#Write_misses = 56

#Penalty_read: lectura del bloque en Memoria Principal
#Penalty_write: escritura en Memoria Principal

$$\text{Time_mem} = 256 * 1\text{ns} + 158 * 200\text{ns} + 56 * 50\text{ns} = 34656\text{ns}$$



Si nos pidieran el tiempo medio de acceso a memoria (**AMAT**):

1ª opción:

$$\text{Average_Time_mem} = \text{Time_mem} / \# \text{Accesses} = 135,4\text{ns}$$

2ª opción:

$$\begin{aligned} \text{Average_Time_mem} = & T_{\text{hit}} + \\ & (\# \text{Read_misses} / \# \text{Accesses}) * \text{Penalty_read} + \\ & (\# \text{Write_misses} / \# \text{Accesses}) * \text{Penalty_write} \end{aligned}$$

$$\# \text{Read_miss_ratio} = (\# \text{Read_misses} / \# \text{Accesses}) = 0,6172$$

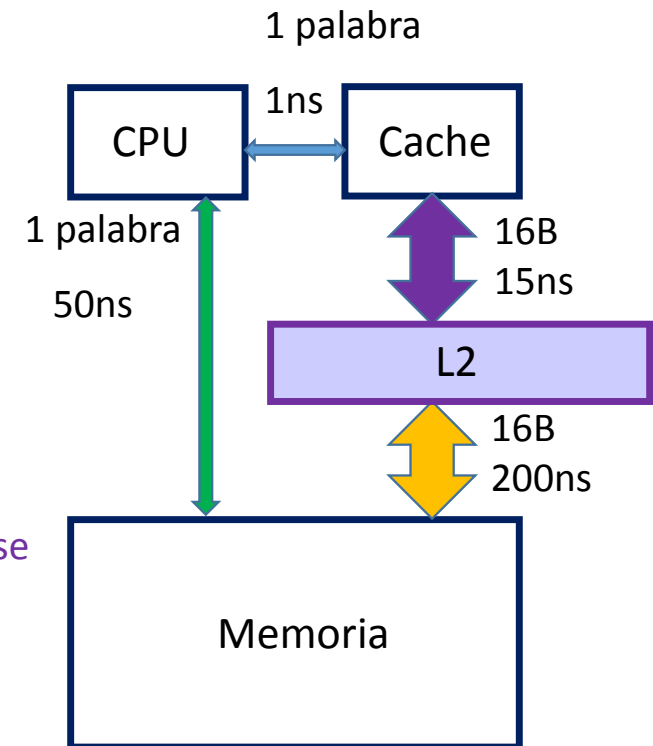
$$\# \text{Write_miss_ratio} = (\# \text{Write_misses} / \# \text{Accesses}) = 0,21875$$

$$\text{Average_Time_mem} = 1\text{ns} + 0,6172 * 200\text{ns} + 0,21875 * 50\text{ns}$$

d) Si se añade al computador una memoria cache de segundo nivel de 1MB y asociatividad 4, con bloques de 16 bytes, con asignación en escritura y actualización diferida y si el tiempo de transferencia de un bloque desde memoria cache de segundo nivel a la de primer nivel son 15ns, ¿cuál es el tiempo necesario para acceder a todas las referencias del programa?

$$\text{Time_mem} = \# \text{Accesses} * T_{\text{hit}} + \# \text{Read_misses} * \text{Penalty_read} + \# \text{Write_misses} * \text{Penalty_write}$$

Puede modificarse por L2!



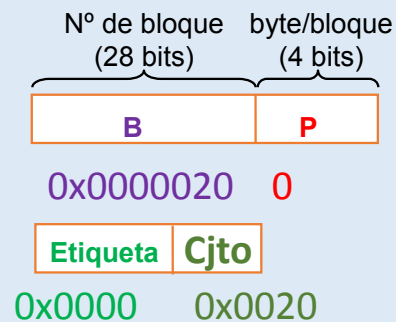
#Penalty_read: lectura del bloque desde L2

#Penalty_write: escritura de palabra en la Memoria Principal (consideramos que esto no cambia)

Estructura de la cache L2

Memoria cache asociativa de 4 vías y de **1MB** con bloques de **16B**
Dónde se almacena en la cache media[0]?

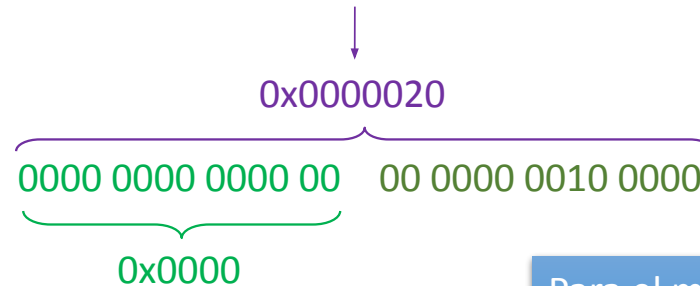
Campos de la dir. De memoria



$$\frac{2^{20} \text{ bytes}}{16 \text{ bytes/bloque}} = 2^{16} \text{ bloques}$$

$$\frac{2^{16} \text{ bloques}}{4 \text{ vías/conjunto}} = 2^{14} \text{ conjuntos}$$

Dir. de memoria: 0x00000200



Para el mismo valor de i nota y media se
almacenan en diferentes conjuntos
No hay conflictos!

Accesos y fallos a L2 (1)

Sólo los fallos de L1 acceden a la L2

```
for i=0 to i=7 and for i=64 to 127  
  a = nota[i]*media[i]
```

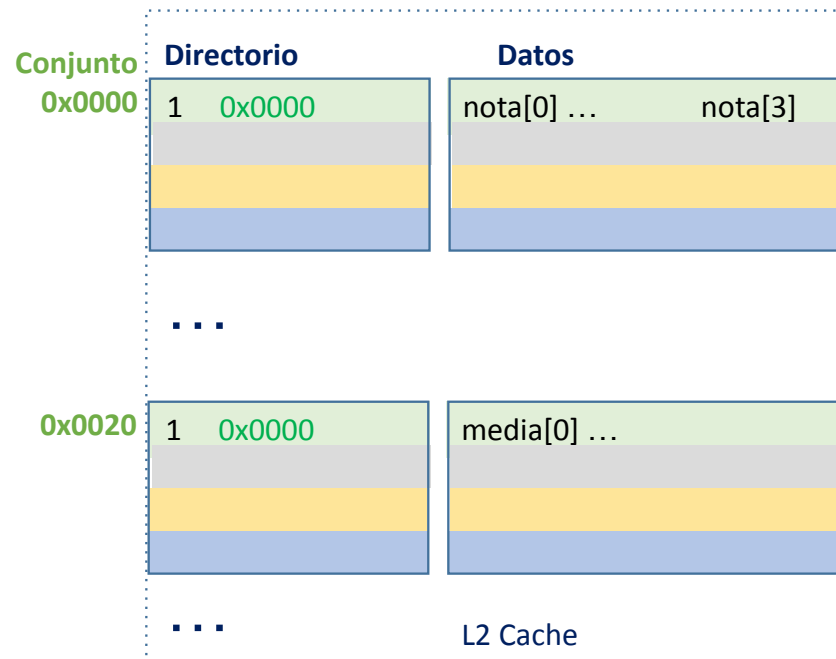
i=0

1. Lectura `nota[0]`

L1 fallo de cahe
L2 fallo de cache (1er acceso)

2. Lectura `media[0]`

L1 fallo de cache
L2 fallo de cache (1er acceso)



Accesos y fallos a L2 (2)

Sólo los fallos de L1 acceden a la L2

```
for i=0 to i=7 and for i=64 to 127  
  a = nota[i]*media[i]
```

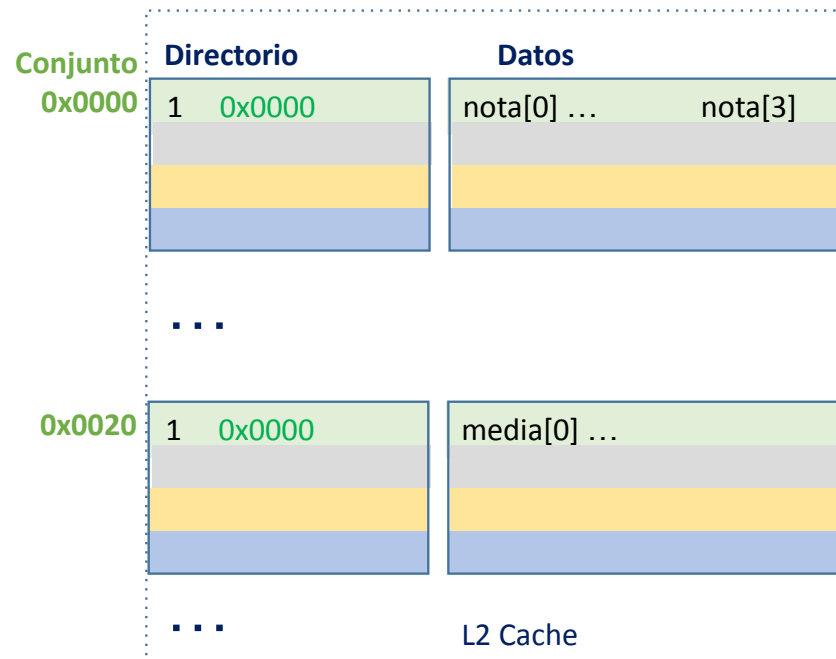
i=1

1. Lectura `nota[1]`

L1 fallo de cache
L2 acierto en la cache

2. Lectura `media[1]`

L1 fallo de cache
L2 acierto en la cache



Iteraciones: 72

Accesos: 2 por iteración

Fallos: 2 cada 4 iteraciones

Accesos y fallos a L2 (3)

Sólo los fallos de L1 acceden a la L2

for i=8 to 63

nota[i] = 0,5*media[i]

i=8

1. Lectura media[8]

L1 fallo de cache

L2 fallo de cache (1er acceso)

2. Escritura nota[8]

L1 fallo de cache: no-write allocate

L2 fallo de cache

Iteraciones: $64 - 8 = 56$

Accesos: (i = 8, 12, 16...) 1 lectura
+ 1 escritura por iteración

Fallos: 1 fallo de lectura cada 4
iteraciones + 1 fallo de escritura
por iteración

1 fallo de lectura por bloque.

Todas las escrituras son fallos (los elementos se escriben en memoria).

#Read_misses * Penalty_read (time for all reads to L2):

$L2_Reads_time = \# L2_reads * L2_Hit_time + \#L2_Read_misses * L2_Penalty_read$

Iteraciones: $64 + 8 = 72$

Accesos: 2 por iteración

Fallos: 2 cada 4 iteraciones

Iteraciones: $64 - 8 = 56$

Accesos: 1 lectura + 1 escritura por iteración

Fallos: 1 fallo de lectura cada 4 iteraciones +
1 fallo de escritura por iteración

$\# L2_reads = \# L1_Read_misses = 2 * 72 + 56/4 = 158$

$\# L2_Read_misses = (2 * 72)/4 + 56/4 = 50$

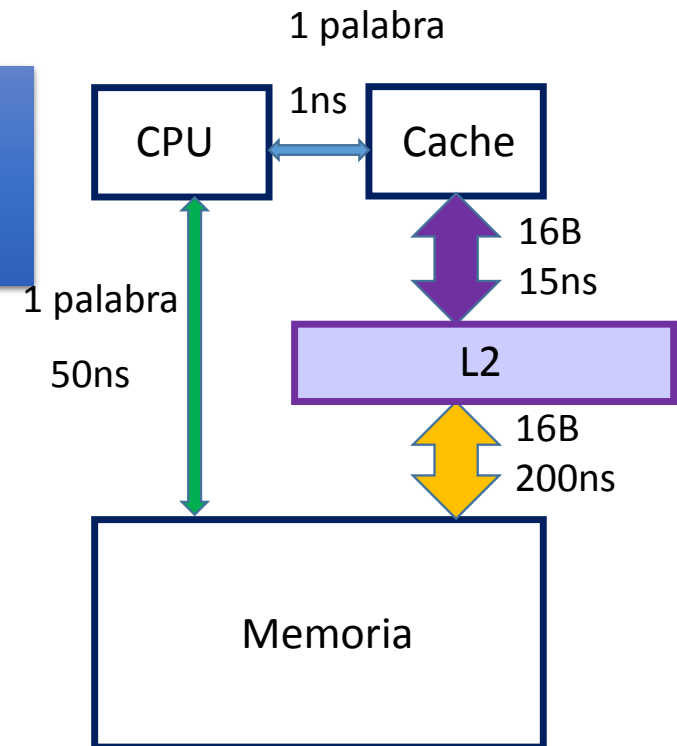
#Read_misses * Penalty_read (time for all reads to L2):
 $L2_Reads_time = \# L2_reads * L2_Hit_time +$
 $\#L2_Read_misses * L2_Penalty_read$

L2_reads = 158
L2_Read_misses = 50

L2_Hit_time = 15ns
#L2_Penalty_read = 200ns

$L2_Reads_time = 158 * 15ns + 50 * 200ns =$
12370ns

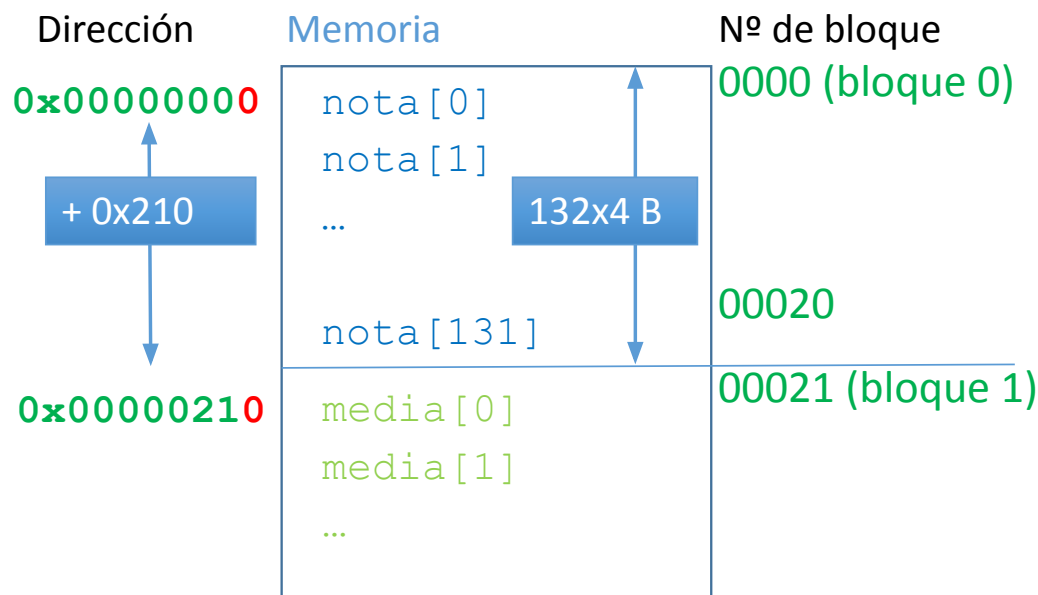
$Time_mem = 256 * 1ns + 12370ns + 56 * 50ns = 15426ns$



Optimizaciones de código

1. **Alargamiento de arrays:** **añadir 1 bloque vacío** al primer vector (nota) para que el segundo vector se almacene en bloques de cache distintos.

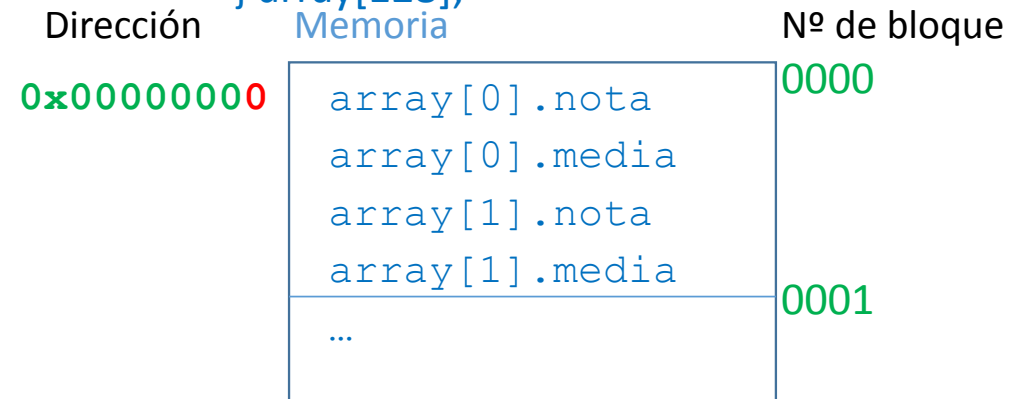
```
int nota[132];
```



Los fallos de lectura se reducen a los de inicio (50)

2. **Fusión de arrays:** **almacenamiento de las mismas posiciones de distintos vectores** en posiciones de memoria contiguas.

```
struct fusion{  
    int nota;  
    int media;  
} array[128];
```



Los fallos de lectura se reducen a los de inicio (64) y los de escritura desaparecen